

ChipMEM: Verification-Grounded Memory for EDA Agents

Abdulrahman AlRabah^{1*} Joshua Mabry² Dilek Hakkani-Tür¹ Abdussalam Alawini¹
 Hamid Shojaei³ Kartik Hegde³ Sandesh Adhikary³
¹University of Illinois Urbana-Champaign ²NVIDIA ³Cadence

Abstract

Large language model (LLM)-based agents use Electronic Design Automation (EDA) tools to generate and revise register-transfer-level (RTL) designs under synthesis and verification feedback. Recent methods learn from this feedback by distilling reusable skills from execution traces or by training on rewards derived from EDA-tools. Both methods are typically evaluated on the tasks that produced the experience. Repeated access to benchmark feedback on the same task can reward task-specific revision rather than creating reusable knowledge that transfers. We introduce ChipMEM, a verification-grounded memory layer for EDA agents. It combines cross-task procedural memory with within-trajectory statistical guidance. Its procedural component distills and stores a skill only after it passes synthesis, simulation, or formal checks, rather than relying on model self-assessments. A Bayesian component maintains hierarchical Beta estimates over tool-call outcomes and ranks recovery strategies that succeeded under comparable errors. A common adapter applies the same memory interface to RTL optimization and testbench-generation agents while preserving each domain’s tools and acceptance criteria. We measure performance on training tasks and evaluate whether learned skills transfer to unseen tasks. On RTLRewriter-Bench, under matched model and tool settings, ChipMEM produces equivalence-passing outputs on 39/54 scored designs versus 35/54 without memory; on the 49-design short suite, mean area improvement is 8.69% versus 5.66%. On held-out CVDP tasks, ChipMEM with a frozen procedural library achieves 20/20 accepted outcomes versus 18/20 without memory in a single evaluation per setting.

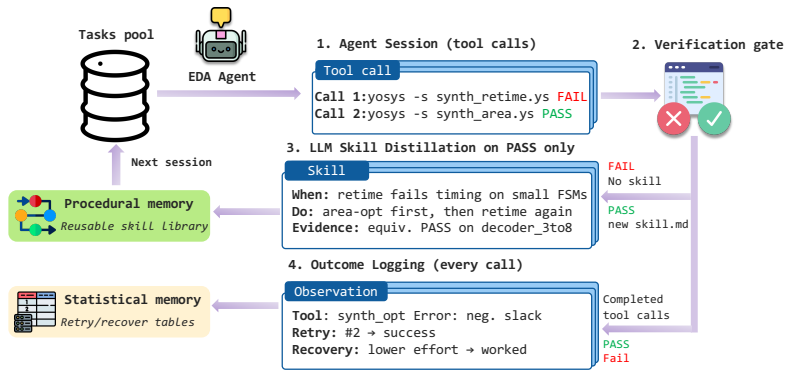


Figure 1: ChipMEM’s seed data come from EDA-agent sessions evaluated by domain tools. The verification gate stores a reusable procedural skill only after a passing final artifact, while statistical memory records every completed tool-call outcome to support retry and recovery decisions in later actions and sessions. Figure 2 shows the full system architecture, including skill retrieval and use.

*Corresponding author: alrabah2@illinois.edu

1 INTRODUCTION

Large language models now support several stages of chip design and verification. They can generate register transfer level (RTL) code, revise hardware descriptions, and use feedback from design tools to correct their outputs [Liu et al., 2023, Lu et al., 2024, Pinckney et al., 2025, Yu et al., 2026]. Recent systems also equip agents with reusable skills distilled from prior tool guided sessions [Du and Pinckney, 2026, Fang et al., 2026]. This approach reduces the need to encode every lesson in a prompt. Continual learning then depends on whether the agent can discover reliable skills from verified work and reuse them on later tasks. This problem matters in chip development because early RTL decisions shape later design quality. Choices about bit width, resource sharing, and state encoding affect power, performance, and area, yet many useful changes require engineers to restructure the RTL source instead of relying on synthesis alone [Lu et al., 2026, Yao et al., 2024]. The revised RTL must then pass simulation and equivalence checks before it can move forward. These iterations are slow, and the lessons learned often remain with individual engineers instead of becoming reusable knowledge. Automation is therefore essential.

These challenges have motivated a rapidly growing body of research on large language models for hardware design. Such models can reason over hardware descriptions, propose candidate rewrites in seconds, and iteratively refine them under feedback from EDA tools. Existing efforts have explored both parametric approaches — adapting model weights through supervised fine-tuning or reinforcement learning [Chen et al., 2026, Shi et al., 2025, Zhou et al., 2026] — and nonparametric approaches that equip frozen models with curated skills, search strategies, or tool-in-the-loop feedback [Arnold et al., 2026, Ping et al., 2026].

Prior work suggests that agents can self-improve by retaining skills, reflections, and strategies from earlier trajectories [Wang et al., 2023, Shinn et al., 2023, Ouyang et al., 2026]. Recent EDA systems have begun incorporating similar forms of tool-grounded experience. However, these methods are typically evaluated on the same task or task family that produced the memory, leaving unresolved whether stored skills remain relevant *across* tasks and *unseen* designs. Such memory is useful if the agent is faced with the same task again, but the agent must once again start from scratch on brand new tasks. Moreover, naively exposing the agent to memories cultivated from dissimilar tasks has limited utility and may even be counterproductive; the optimal skills for one design may actively diminish performance on a different design. This gap motivates a memory layer whose contents are certified by EDA tools and whose value is measured through transfer beyond the tasks that produced them.

To address this gap, we propose ChipMEM², a verification-grounded memory layer for cross-task transfer that combines **procedural memory** and **statistical memory**. The **procedural memory** captures complete agent trajectories and distills them into reusable skills to be retrieved for subsequent tasks. These skills are collected and updated according to outcomes from verifiable evaluators for specific tasks, e.g. EDA tools for synthesis and verification. A successful strategy may create a new skill or refine an existing one, while a verified failure may be retained as guidance against repeating an ineffective action. Moreover, the skills are indexed by an embedding derived from the design RTL. Given a new task, similarities with respect to these embeddings are used to retrieve the skills most likely to be related to and beneficial for the current task; the central hypothesis being that similar designs will likely have transferable skills. The **statistical memory** complements these trajectory-derived lessons with a Bayesian model for continual learning through current and historical successes and failures, estimating the probability that a tool strategy will succeed in the current execution context and advising the agent when an unchanged retry is unlikely to be productive. Instead of solely relying on the agent’s intrinsic reasoning to identify the right tools, we equip it with a “data analyst” that produces advice grounded in statistics collected from prior rollouts. With statistical memory, the agent is thus able to begin a session with an informed prior over success likelihoods of tools, and can update this prior with each tool execution.

ChipMEM combines the procedural and statistical memories into a single memory layer through a common agent adapter that records the task representation, retrieved context, execution trajectory, returned artifacts, and domain-specific acceptance criterion, allowing both memory components to support EDA agents without requiring those agents to share the same tools or scoring procedure. This design allows procedural memory to evolve across a sequence of tasks or remain frozen for

²Code: <https://github.com/aalrabah/ChipMEM.git>

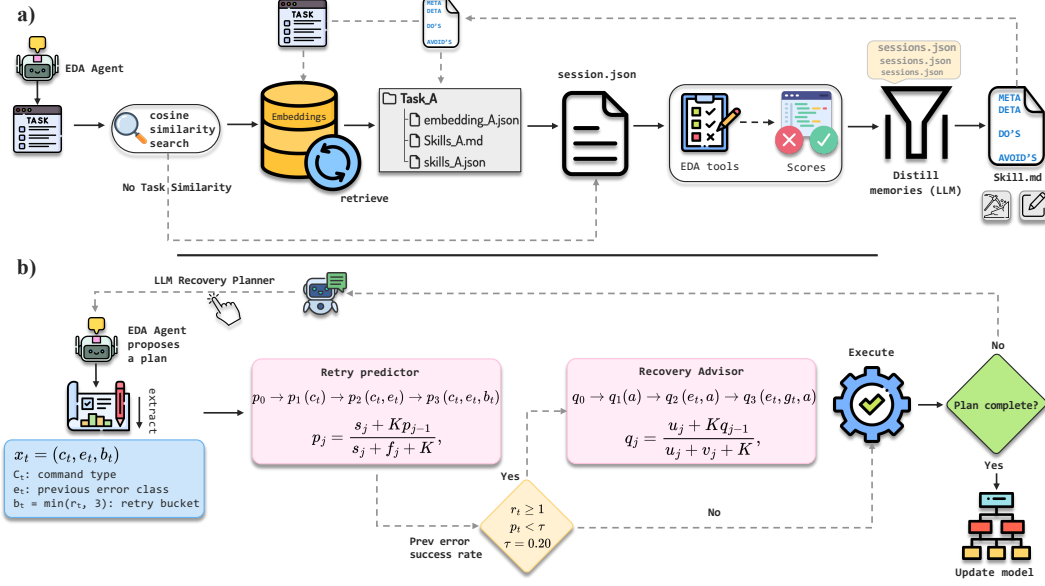


Figure 2: ChipMEM workflow. (a) Procedural memory retrieves skills by task similarity, supplies them to the EDA agent, and stores a distilled skill only after a verified PASS. (b) Statistical memory updates from each completed tool call, estimates retry success, and invokes a recovery advisor and bounded LLM plan when the estimate falls below $\tau = 0.20$.

evaluation on unseen designs, while statistical memory summarizes execution outcomes and provides probabilistic guidance during tool use.

We summarize our primary contributions as follows:

1. ChipMEM combines procedural memory that extracts reusable skills from tool-scored executions with Bayesian statistical memory that accumulates tool-level successes and failures to guide retries and recovery.
2. ChipMEM provides an agent-agnostic adapter between an EDA agent, its tools, and its evaluation harness. The adapter supplies retrieved skills and recovery guidance, records execution trajectories, and returns artifacts for domain-specific evaluation without modifying the underlying agent or tools.
3. We evaluate ChipMEM against memory-off baselines across multiple EDA tasks and benchmarks, examining verified outcomes, interaction cost, and the effects of each memory component.
4. We separately evaluate transfer to unseen tasks using a fixed, read-only procedural library, testing whether prior experience remains useful without adding new skills during evaluation.

We evaluate ChipMEM on the RTL-OPT [Lu et al., 2026] and RTLRewriter [Yao et al., 2024] benchmarks for PPA optimization, and on CVDP tasks for testbench generation [Pinckney et al., 2025]. The evaluation tests whether skills distilled from verified trajectories transfer to later tasks and unseen designs, and whether accumulated tool outcomes improve decisions during execution.

2 RELATED WORK

Skill memory and design-aware retrieval. There is a growing body of work that equips frozen agents with procedural memory, storing experience from past episodes as reusable artifacts and retrieving it to guide later tasks. Voyager [Wang et al., 2023] builds an executable skill library from successful episodes, Reflexion [Shinn et al., 2023] turns execution feedback into verbal episodic

memory, ReasoningBank [Ouyang et al., 2025] distills transferable strategies from both successful and failed trajectories, and SkillOpt [Yang et al., 2026] refines a skill document over a frozen model without weight updates. In chip design, however, the relevance of a stored strategy depends on the target specification, RTL structure, tool context, and failure mode, so a skill that helps one design may be ineffective or harmful on another. This makes retrieval as important as skill extraction and motivates evaluating memory the task that produced it. ChipMEM extends this direction by integrating trajectory-distilled procedural memory for cross-task transfer with Bayesian statistical memory for within-session guidance, while grounding both in outcomes produced by EDA tools.

Tool-grounded RTL optimization. RTL-OPT [Lu et al., 2026] and RTLRewriter [Yao et al., 2024] evaluate LLM-driven RTL optimization under both implementation-quality and functional-equivalence criteria. Recent methods then fold EDA feedback into optimization in different ways. Reinforcement learning drives a gated toolchain reward into model weights [Chen et al., 2026]. Evolutionary and agentic search explore candidate rewrites under synthesis feedback [Ping et al., 2026, Arnold et al., 2026, Hsin et al., 2026]. Frozen agents instead evolve reusable skills from execution traces [Du and Pinckney, 2026, Fang et al., 2026, Wang et al., 2026], and test-time training adapts weights to a single design and discards them once it changes [Zhou et al., 2026]. These methods improve how an agent searches or adapts during optimization, whereas ChipMEM turns verified outcomes into reusable procedural and statistical memory that transfers to later tasks and different EDA agents.

LLMs for RTL design and evaluation. VerilogEval [Liu et al., 2023], RTLLM [Lu et al., 2024], and CVDP [Pinckney et al., 2025] provide established benchmarks for RTL generation, completion, and verification. Building on these efforts, ChipMEM extends the evaluation beyond isolated task success to examine whether tool-verified experience remains useful across later tasks and unseen designs, and whether historical outcomes can improve decisions during tool-guided execution.

3 ChipMEM

ChipMEM is an agent-agnostic layer that converts tool-verified execution traces into two complementary memories: **procedural memory** preserves reusable strategies across tasks, while **statistical memory** learns action-level success and recovery patterns within and across sessions. Both components augment the agent context without changing the underlying EDA agent, tools, or domain-specific acceptance gate. Figure 2 summarizes this dual-memory workflow.

Procedural memory. We represent the procedural memory library as key-value map $\mathcal{M}^{\text{proc}} = \{k_i \mapsto (v_i, m_i, h_i)\}_{i=1}^M$. For a given task, the value tuple (v_i, m_i, h_i) consists of the complete skill file v_i , metadata m_i for its domain, task, provenance, and verifier outcomes h_i . For the i -th task, the key $k_i = \phi(z_i) \in \mathbb{R}^d$ is the task embedding obtained by applying an embedding model ϕ to a pre-defined task-artifact z_i . For instance, in our experiments, z_i corresponds to the top RTL file for RTL optimization and the task-definition PROMPT.md file for CVDP tasks. This task definition excludes agent instructions, retrieved skills, tool history, generated files, tests, and scoring artifacts.

The library starts empty and grows across N_{seed} tasks run for R rounds. For the n -th task, we retrieve procedural-memory as value-tuples with keys exhibiting the largest similarity to the current task’s embedding q_n , i.e.,

$$\mathcal{R}_n = \{(v_i, m_i, h_i) \mid i \in \text{TopK}[\text{sim}(q_n, k_j); \tau_{\text{proc}}]\}.$$

Here, $\text{TopK}[\text{sim}(\cdot); \tau_{\text{proc}}]$ is the list of top-k matches with respect to a similarity function sim and a threshold τ_{proc} ; only matches with similarities higher than τ_{proc} are returned. We use cosine-similarity over embeddings as our similarity function $\text{sim}(q_n, k_j)$. Thus $\mathcal{R}_t$ contains at most K complete memory items with the highest task similarity above a threshold τ_{proc} .

Once the n -th task is processed, ChipMEM may expand the procedural memory library by adding a skill for the source task if none exist. Skills are added to the library only if the corresponding task execution obtains a verified PASS from a verifiable domain-specific evaluation harness, not an LLM judge. We can apply the procedural memory library under two modes: evolving and frozen. While the evolving-mode carries verified additions forward to later tasks, the frozen-mode keeps its initial library read-only throughout.

Statistical memory. While procedural memory allows EDA agents to extract high-level strategies to aid their tasks, agents can also benefit from the more structured and narrower-scoped skill of tool-failure-recovery. EDA agents often repeat failed tool calls without reusing recoveries that have succeeded in earlier trajectories. Since EDA-tools can have particularly high latency, repeated tool-call failures can lead to significant delays and wasted token-consumption. Thus, we propose statistical memory that maintains a prior belief over tool-call success based on statistics collected from prior runs, updates the prior based on the current trajectory, and provides advice on tool-retry-success and failure-recovery strategies by learning on a tool-step granularity. The statistical memory component is comprised of two binomial distribution models – the retry predictor and the recovery predictor – encoding the success likelihood of tool-calls and their corresponding failure-recovery actions. As illustrated in Figure 2, the retry predictor informs the agent whether its proposed tool-call is likely to succeed, and the recovery predictor provides the most-successful strategy to recover from that failure.

At the t -th turn in the execution trajectory of an agent, let c_t be tool-call submitted by the agent; further assume that this tool has previously been called r_t times in the same session with the last call resulting in error e_t . If the previous call for c_t succeeded, e_t is set to None; otherwise it is set to the specific error class encountered. For example, in our experiments with the PPA agent, such error classes include `syntax-error`, `unknown-module`, `liberty-missing`, `timing-fail`, and `rc-nonzero`. This provides us with statistics on the likelihood of a tool’s success. Following a tool-call error, we also record the action a_t taken by the agent, and whether the a_t resulted in a successful recovery. For each error class e_t , we thus collect a table of associated recovery actions and their corresponding likelihoods of success.

We define the context of the retry-model via the tuple $x_t = (c_t, e_t, b_t)$, where $b_t = \min(r_t, 3)$. We set $b_t = \min(r_t, 3)$, so its values are 0, 1, 2, and 3, with 3 grouping three or more earlier calls. This saturation prevents sparse evidence from being split across many large retry counts. Starting from the global Beta(1, 1) posterior p_0 , the model adds one conditioning variable at each of three levels. Level p_1 conditions on command type, p_2 adds 1-step history of the previous error class, and p_3 adds prior history: $p_0 \rightarrow p_1(c_t) \rightarrow p_2(c_t, e_t) \rightarrow p_3(c_t, e_t, b_t)$, with

$$p_j = \frac{s_j + Kp_{j-1}}{s_j + f_j + K}, \quad j \in \{1, 2, 3\}, \quad (1)$$

where s_j and f_j are prior successful and failed calls at level j , and the parameter K essentially controls the weight placed on prior history; a lower value of K makes the model more myopic. In our experiments, we use $K = 8$. The final estimate is $P_{\text{retry}}(\text{success} \mid c_t, e_t, b_t) = p_3$. After a failed same-type call, a value below $\tau_{\text{stat}} = 0.20$ activates the recovery model.

For a candidate recovery strategy a and the current tool stage g_t . The recovery hierarchy is

$$q_0 \rightarrow q_1(a) \rightarrow q_2(a, e_t) \rightarrow q_3(a, e_t, g_t).$$

For example, $q_1(\text{inspect_logs})$ estimates how often inspecting logs has recovered a failure across all errors and stages. At each level,

$$q_j = \frac{u_j + Kq_{j-1}}{u_j + v_j + K}, \quad j \in \{1, 2, 3\}, \quad (2)$$

where u_j and v_j are historical successful and failed recoveries, q_{j-1} is the probability inherited from the broader level, and $K = 8$. Hence $P_{\text{recovery}}(a \mid e_t, g_t) = q_3$, and candidate strategies are ranked as

$$\pi(e_t, g_t) = \text{sort}_a P_{\text{recovery}}(a \mid e_t, g_t). \quad (3)$$

When activated, the recovery model passes its ranked strategies to an LLM recovery planner, which returns a bounded advisory plan to the agent. The ranking is advisory; each completed tool call atomically persists retry and recovery evidence for the next decision, including the next decision in the same session.

Algorithm. Algorithm 1 summarizes ChipMEM across ordered sessions. Here D_n is the task artifact, $\mathcal{A}$ is the EDA agent, $\mathcal{T}$ is its tool set, $\mathcal{V}$ is the deterministic domain evaluation harness, q_n is the task embedding, $\mathcal{R}_n$ is the retrieved skill set, H_n is the execution trajectory, and y_n is the final harness verdict. **statistical memory** updates after every tool call. Only a passing trajectory can add a

skill to procedural memory. The algorithm shows evolving mode. Frozen mode starts from a fixed library and skips the skill storage step.

Algorithm 1 procedural memory and statistical memory across sessions

Require: Tasks $\{D_n\}_{n=1}^N$, agent $\mathcal{A}$, EDA tools $\mathcal{T}$, domain evaluation harness $\mathcal{V}$, embedder ϕ

- 1: $\mathcal{M}^{\text{proc}} \leftarrow \emptyset$; initialize $\mathcal{M}^{\text{stat}}$ with Beta(1, 1) priors $\triangleright$ procedural library; Bayesian state
- 2: **for** $n = 1, \dots, N$ **do** $\triangleright n$: session; N : total sessions
- 3: $q_n \leftarrow \text{EMBEDTASK}(\phi, D_n)$ $\triangleright q_n$: task embedding
- 4: $\mathcal{R}_n \leftarrow \text{RETRIEVESKILLS}(\mathcal{M}^{\text{proc}}, q_n, \tau_{\text{proc}})$ $\triangleright \mathcal{R}_n$: top K skill payloads; τ_{proc} : threshold
- 5: Give $(D_n, \mathcal{R}_n)$ to $\mathcal{A}$; initialize $H_n \leftarrow \emptyset$ $\triangleright H_n$: trajectory
- 6: **while** session n is active **do**
- 7: $\mathcal{A}$ selects an action and calls a tool in $\mathcal{T}$ $\triangleright$ **Act**
- 8: Append the action and returned outcome to H_n
- 9: $\mathcal{M}^{\text{stat}} \leftarrow \text{BAYESUPDATE}(\mathcal{M}^{\text{stat}}, \text{latest tool outcome})$ $\triangleright x_t = (e_t, c_t, b_t)$; retry model
- 10: **if** a repeated failure has $P_{\text{retry}} < \tau_B$ **then** $\triangleright$ retry probability; nudge threshold
- 11: Advise $\mathcal{A}$ using $\pi(e_t, g_t)$ $\triangleright$ recovery model; e_t : error; g_t : stage
- 12: **end if**
- 13: **end while**
- 14: $y_n \leftarrow \text{RUNDOMAINHARNESS}(\mathcal{V}, D_n, H_n)$ $\triangleright y_n$: final harness verdict
- 15: **if** $y_n = \text{PASS}$ **then**
- 16: $v_n \leftarrow \text{DISTILLPROCEDURES}(H_n)$ $\triangleright$ procedural distiller; v_n : skill
- 17: $\mathcal{M}^{\text{proc}} \leftarrow \text{STORESKILL}(\mathcal{M}^{\text{proc}}, q_n, v_n)$ $\triangleright q_n$: index; v_n : skill
- 18: **else**
- 19: Leave $\mathcal{M}^{\text{proc}}$ unchanged
- 20: **end if**
- 21: **end for**

Agent adapter and verification. ChipMEM connects to each EDA domain through a common adapter. For task D_n , the adapter gives $\mathcal{A}$ the original task, retrieved skills $\mathcal{R}_n$, and any Bayesian recovery advice. It exposes the domain tools, records each tool action and result in H_n , and returns the final artifact to $\mathcal{V}$. Here $\mathcal{V}$ is a deterministic domain evaluation harness, not an LLM judge. PPA acceptance requires successful synthesis, functional equivalence, and a positive audited improvement. CVDP testbench generation requires the hidden simulation, coverage, and mutation checks to pass. In evolving mode, a verified PASS may add a procedural skill. Frozen mode leaves procedural memory unchanged. A FAIL or INVALID verdict creates no skill, while statistical memory still records completed tool outcomes.

4 Experimental Setup

Tasks The CVDP study uses 40 testbench tasks, with 20 training tasks and 20 held-out tasks split evenly between CID012 stimulus generation and CID013 checker generation [Pinckney et al., 2025]. RTL-OPT contains 38 design-level RTL optimization tasks [Lu et al., 2026], while RTLRewriter-Bench contains 59 available cases, 54 short and five long [Yao et al., 2024]. OpenTitan evaluates ten IP blocks over five attempts per mode [Meza et al., 2023]. Additionally, we also performed evaluations on a custom set of 20 open-sourced designs (see Appendix for details).

Implementation details. We use GPT-5.5 [OpenAI, 2026] for RTL-OPT, RTLRewriter-Bench, the custom-design PPA study, and CVDP CID012 and CID013. We use Qwen3.8-27B [Qwen Team, 2026] for OpenTitan. Procedural memory retrieval uses text-embedding-3-large for the first group and Qwen/Qwen3-Embedding-0.6B for OpenTitan. Reasoning is set to medium for all experiments. OpenTitan uses at most 40 tool steps, a 600 second model timeout, and a 7200 second session limit. Retrieval returns at most two skills at similarity threshold 0.6. The open model and its embedding model run on an NVIDIA A100 80GB GPU [NVIDIA Corporation, 2020]. Appendix A.5 reports the detailed resource measurements.

Evaluation metrics. Equivalence pass rate and strict gate pass rate are binary measures of functional correctness and complete task acceptance. For PPA, area, power, and timing gains are mea-

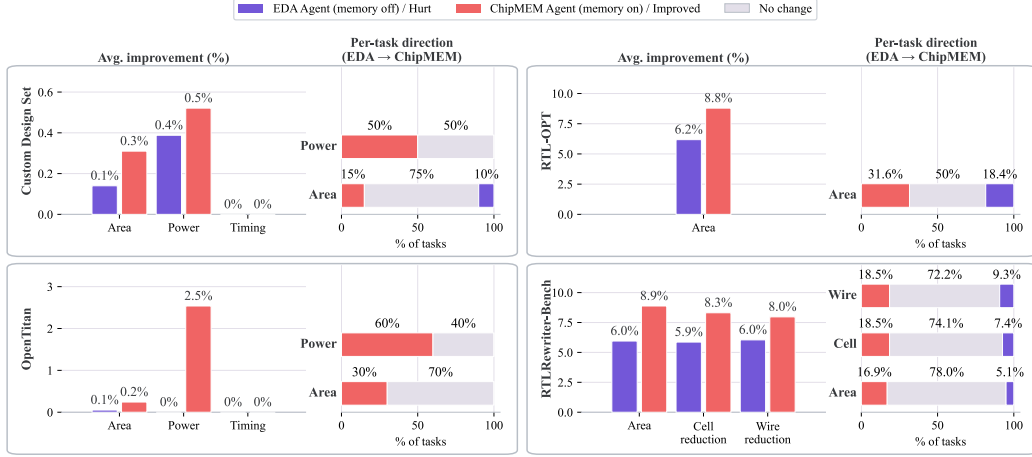


Figure 3: Audited PPA outcomes across the custom-design PPA suite, OpenTitan, RTL-OPT, and RTLRewriter-Bench. Vertical bar graphs compare baseline EDA-Agent with no memory (purple) against ChipMEM (red). Horizontal bars show the per-task direction of change under ChipMEM as improved, unchanged, or hurt. RTLRewriter means use 54 scored cases after excluding five zero-cell cases. For, RTL-OPT we select the best valid result per design across three result sets and reports the strongest demonstrated capability.

sured relative to the original design after synthesis and equivalence. Failed, invalid, unproven, unchanged, and nonimproving outputs receive zero gain. RTLRewriter additionally reports geometric mean cell and wire ratios relative to the baseline, where values below one indicate reduction. For CID012 and CID013, task pass rate is the fraction of generated testbenches that pass every hidden Xcelium/IMC test. For ordered passes, cumulative Pass@ k is the fraction of tasks solved by pass k and is reported as a sequential curve rather than an independent sampling estimate.

An EDA agent works on one design over many turns. It reads and edits files, calls design tools, inspects failures, and revises its output before the final evaluation. The PPA agent can call synthesis, simulation, timing, and equivalence tools. The simulation agent can compile and run testbenches. The CVDP simulation set contains 20 training tasks and 20 unseen tasks. CID012 stimulus generation and CID013 checker generation each contribute 10 training tasks and 10 unseen tasks. The unseen tasks are disjoint from training. The agent sees the task files and design context, while hidden tests and reference artifacts remain outside its workspace.

PPA metric scope. The PPA studies use two evaluation scopes. We began with RTL-OPT and RTLRewriter-Bench as first-pass area-optimization studies using Yosys for synthesis and equivalence and ABC for technology mapping [Wolf and Glaser, 2013, Brayton and Mishchenko, 2010]. This fast, reproducible loop let us verify that the agent workflow, equivalence gate, and measurement methodology operated correctly before moving to the broader Custom Design Set and OpenTitan flows. The first-pass runs measure mapped area, and RTLRewriter-Bench also records cell and wire counts. Because we did not run a matched characterized downstream power or timing flow for these two benchmarks, we do not report unmeasured power or performance values. The later Custom Design Set and OpenTitan studies report full PPA metrics, including area, power, and timing. Figure 3 presents the first-pass area-optimization and full-PPA results together under their respective measurement scopes.

Baselines and ablations. Figure 3 and Appendix Table 2 compare *EDA agent*³ with *EDA agent* + *ChipMEM*. *EDA agent* is the same domain agent with memory disabled. It receives no retrieved skills, writes no skills, and receives no Bayesian guidance. *EDA agent* + *ChipMEM* uses the same model, task, prompt, tools, and budget with memory enabled. In evolving mode, *EDA agent* + *Chip-*

³The EDA Agent is Cadence ChipStack 2.0, a proprietary system actively used in production by leading chip-design companies; all GPT-5.5 experiments were run on this agent.

Table 1: Five-design RTL-OPT memory study ($n = 5$). Tokens report aggregate input/output tokens. Tool/LLM reports tool calls and successful agent LLM turns. Time is summed session wall time. Best values are bold.

Memory component ablation and loaded state replay ($n = 5$)						
Mode	PASS	Gain	Tokens in/out	Tokens /PASS	Tool /LLM	Time
Memory OFF	3	8.41%	1.82M/150k	657k	183/191	111.5m
Procedural only	4	10.13%	1.67M/96k	442k	173/184	66.7m
Bayesian only	4	8.60%	1.29M /84k	344k	135/147	67.7m
Procedural + Bayesian	5	10.36%	1.49M/ 72k	312k	119/133	49.2m

MEM updates the skill library after verified passing tasks. In frozen mode, *EDA agent + ChipMEM* reads a fixed library during unseen task evaluation. These settings provide the whole-system comparison between Memory OFF and full ChipMEM. Separately, on five RTL-OPT designs, we compare Memory OFF, procedural-only, and Bayesian-only under matched settings, while the procedural-plus-Bayesian loaded-state replay reports combined system capability.

5 Results and Discussion

Cross benchmark interpretation. Figure 3 shows that, on the two public benchmarks, ChipMEM increases mean area improvement from 6.18% to 8.79% on RTL-OPT and from 5.95% to 8.88% on RTLRewriter-Bench. It also raises RTLRewriter-Bench cell and wire reductions from 5.9%/6.0% to 8.3%/8.0%. The per-design results include 12 area improvements on RTL-OPT and 10 on RTLRewriter-Bench, showing that the gains extend across distinct public RTL optimization tasks. In the full-PPA studies, the Custom Design Set increases mean area and power improvement from 0.14%/0.39% to 0.31%/0.52%. OpenTitan increases mean area and power improvement from 0.05%/0.00% to 0.24%/2.54%, while timing remains stable. Table 3 further shows that OpenTitan full-PPA gate passes rise from 1/10 to 4/10 with equivalence maintained at 6/10 in both modes. Together, these results show that ChipMEM improves both public area-optimization benchmarks and the broader full-PPA design flows.

OpenTitan interaction cost. Figure 4 reports lower means for ChipMEM on numerous resource measures. Mean LLM calls fall from 29.4 to 26.1, mean tool calls from 27.4 to 23.3, and mean tool time from 420.2 to 369.8 seconds, corresponding to reductions of 11.2%, 15.0%, and 12.0%. Input tokens decrease by 6.8%, and total tokens decrease by 3.4%. Across the 50 sessions, ChipMEM has lower average LLM calls, tool calls, input and total tokens, and tool time.

Memory ablation. We evaluate four memory configurations on the same RTL-OPT designs ($n = 5$) with matched model, task order, RTL inputs, tools, seed, and execution limits.

Table 1 reports strict PASS counts, audited area gain, and interaction cost. Procedural and Bayesian each produce four strict passes, compared with three for Memory OFF. The combined state produces five strict passes, the highest mean audited gain, and the lowest tokens per accepted design. It records the capability of the full system after both memories accumulated prior experience on the same design sequence.

Held-out PPA skill retrieval. Figure 5 shows representative summaries of skills retrieved during one-shot PPA evaluations of 36 held-out designs with evolving procedural memory. The 50 skill–design retrievals comprised 25 *RTL Optimization*, 21 *Flow & Validation*, and 4 *Design Setup* retrievals, indicating that the agent most often received concrete RTL lessons and procedures for obtaining trustworthy synthesis results. Cohort A produced 38 skill–design retrievals, compared

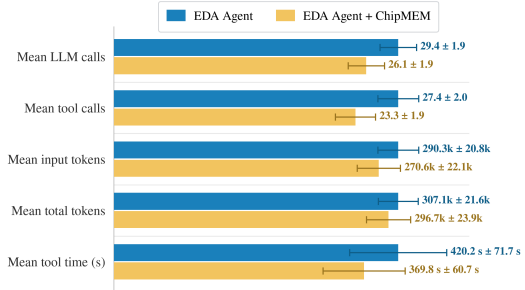


Figure 4: OpenTitan interaction cost over 50 sessions per mode, comprising ten designs and five attempts per design. Bars and labels report mean $\pm$ standard error for LLM calls, tool calls, input and total tokens, and tool time.

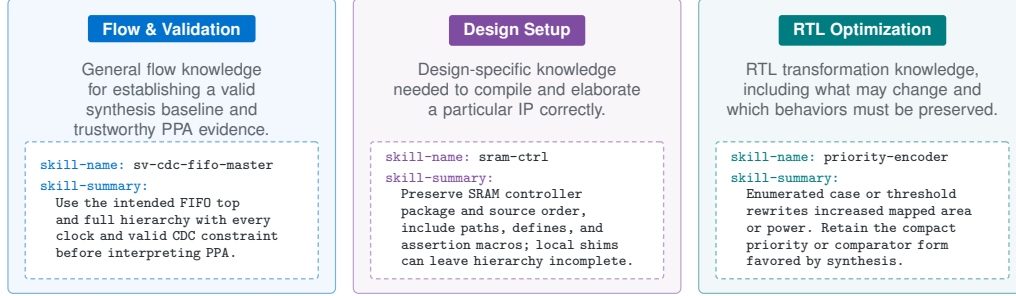


Figure 5: Procedural skill types and representative stored skill summaries. *Flow & Validation* captures general synthesis and evidence-quality procedures, *Design Setup* preserves design-specific compilation knowledge, and *RTL Optimization* records concrete transformations and their guardrails; dashed boxes show one stored example for each type.

with 12 in cohort B (Appendix A.2.2). Beyond its larger size, cohort A contains more FIFOs, arbiters, and peripheral controllers similar to designs represented in the skill library, whereas cohort B contains more heterogeneous processor, memory, and high-speed interconnect blocks.

Table 2: Held-out transfer on ten unseen tasks per category, evaluated once per setting with frozen, read-only memory.

<i>Held-out transfer (frozen, read-only memory)</i>			
Held-out task category	<i>EDA agent</i>	<i>EDA agent + ChipMEM</i>	Change
CID012 stimulus	10/10 (100%)	10/10 (100%)	0 pp
CID013 checker	8/10 (80%)	10/10 (100%)	+20 pp
Overall	18/20 (90%)	20/20 (100%)	+10 pp

Held-out CVDP transfer. Table 2 evaluates frozen memory on unseen tasks. CID012 preserves 10/10 performance in both settings. CID013 increases from 8/10 to 10/10 and raises the overall result from 18/20 to 20/20. Because the library is read-only, the two additional passes reflect transfer from previously learned procedural memory. The result shows that stored experience improves checker generation while preserving perfect stimulus generation.

What the memory learns. The held-out retrievals show that ChipMEM preserves cross-design RTL knowledge, not only instructions for operating EDA tools. This distinction matters: completing the synthesis flow establishes that a result can be evaluated, whereas improving PPA requires transferable design reasoning. Future work should isolate the contribution of individual retrieved skills to flow completion and PPA improvement through paired, skill-level ablations.

6 Limitations

ChipMEM uses a fixed top two retrieval setting at threshold 0.6. The component and frozen transfer studies cover five RTL-OPT designs and ten unseen CVDP tasks per category. Future work should test broader retrieval settings, additional EDA tasks, and controlled runtime environments.

7 Conclusion

ChipMEM converts tool verified EDA experience into reusable procedural and statistical memory. It improves accepted outcomes without changing the underlying agent. In the OpenTitan evaluation, it used fewer LLM and tool calls, fewer input and total tokens, and less tool time on average. The SWE-bench Pro [Deng et al., 2025] pilot in Appendix A.7 extends the evaluation to software engineering and motivates testing ChipMEM across additional domains. Future work should test larger memories and broader cross-domain transfer.

References

- Felix Arnold, Ryan Amaudruz, Dimitrios Tsaras, Renzo Andri, and Lukas Cavigelli. Rtlscout: Joint agentic code and synthesis optimization for efficient digital circuits. *arXiv preprint arXiv:2606.06530*, 2026.
- Robert Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification*, pages 24–40. Springer, 2010. doi: 10.1007/978-3-642-14295-6_5.
- Zhirong Chen, Kaiyan Chang, Zhuolin Li, Cangyuan Li, Xinyang He, Chujie Chen, Mengdi Wang, Haobo Xu, Yinhe Han, Huawei Li, et al. Chipseek: Optimizing verilog generation via eda-integrated reinforcement learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 25180–25201, 2026.
- Xiang Deng et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- Zijian Du and Nathaniel Pinckney. Trace2skill: Verifier-guided skill evolution for long-context eda agents. *arXiv preprint arXiv:2605.21810*, 2026.
- Wenji Fang, Yao Lu, Shang Liu, Jing Wang, Ziyang Guo, Junxian He, Fengbin Tu, and Zhiyao Xie. Dr. rtl: Autonomous agentic rtl optimization through tool-grounded self-improvement. *arXiv preprint arXiv:2604.14989*, 2026.
- Wei-Po Hsin, Ren-Hao Deng, Yao-Ting Hsieh, En-Ming Huang, and Shih-Hao Hung. Evolve: Evolutionary search for llm-based verilog generation and optimization. *arXiv preprint arXiv:2601.18067*, 2026.
- Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for verilog code generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–8. IEEE, 2023.
- Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. Rtlm: An open-source benchmark for design rtl generation with large language model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 722–727. IEEE, 2024.
- Yao Lu, Shang Liu, Hangan Zhou, Wenji Fang, Qijun Zhang, and Zhiyao Xie. A new benchmark for the appropriate evaluation of rtl code optimization. *arXiv preprint arXiv:2601.01765*, 2026.
- Andres Meza, Francesco Restuccia, Jason Oberg, Dominic Rizzo, and Ryan Kastner. Security verification of the opentitan hardware root of trust. *IEEE Security & Privacy*, 21(3):27–36, 2023. doi: 10.1109/MSEC.2023.3251954.
- NVIDIA Corporation. Nvidia a100 tensor core gpu. <https://www.nvidia.com/en-us/data-center/a100/>, 2020.
- OpenAI. Gpt-5.5 model. <https://developers.openai.com/api/docs/models/gpt-5.5>, 2026.
- Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025.
- Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, volume 2026, pages 94327–94354, 2026.
- Nathaniel Pinckney, Chenhui Deng, Chia-Tung Ho, Yun-Da Tsai, Mingjie Liu, Wenfei Zhou, Brucek Khailany, and Haoxing Ren. Comprehensive verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on rtl design and verification. *arXiv preprint arXiv:2506.14074*, 2025.

- Heng Ping, Peiyu Zhang, Zhenkun Wang, Shixuan Li, Anzhe Cheng, Wei Yang, Paul Bogdan, and Shahin Nazarian. Poet: Power-oriented evolutionary tuning for llm-based rtl ppa optimization. *arXiv preprint arXiv:2603.19333*, 2026.
- Qwen Team. Qwen3.8-27b. <https://huggingface.co/Qwen/Qwen3.8-27B>, 2026.
- Jiahe Shi, Zhengqi Gao, Ching-Yun Ko, and Duane Boning. Earl: Entropy-aware rl alignment of llms for reliable rtl code generation. *arXiv preprint arXiv:2511.12033*, 2025.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Yaoliang Wang, Qi Shi, ShangZhan Li, Qingguo Hu, Xinyu Yin, Bo Guo, Xu Han, Maosong Sun, and Jinsong Su. Veriagent: A tool-integrated multi-agent system with evolving memory for ppa-aware rtl code generation. *arXiv preprint arXiv:2603.17613*, 2026.
- Clifford Wolf and Johann Glaser. Yosys – a free verilog synthesis suite. In *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*, 2013. URL <https://yosyshq.net/yosys/files/yosys-austrochip2013.pdf>.
- Yifan Yang, Ziyang Gong, Weiwan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, et al. Skillopt: Executive strategy for self-evolving agent skills. *arXiv preprint arXiv:2605.23904*, 2026.
- Xufeng Yao, Yiwen Wang, Xing Li, Yingzhao Lian, Ran Chen, Lei Chen, Mingxuan Yuan, Hong Xu, and Bei Yu. Rtlrewriter: Methodologies for large models aided rtl code optimization. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pages 1–7, 2024.
- Cunxi Yu, Chenhui Deng, Nathaniel Pinckney, and Bruce Khailany. Agentic hardware design as repository-level code evolution. *arXiv preprint arXiv:2606.28279*, 2026.
- Peilong Zhou, Zhirong Chen, Cangyuan Li, Haoyu Gao, Kaiyan Chang, Ziming Qu, and Ying Wang. Alpha-rtl: Test-time training for rtl hardware optimization. *arXiv preprint arXiv:2606.05253*, 2026.

ChipMEM: Verification-Grounded Memory for EDA Agents

Supplementary Material

Appendix

Table of Contents

A.1 Per-Design PPA Results	14
A.2 Held-Out A and B: Per-Design PPA Results	15
A.3 OpenTitan Ten-Design PPA Audit	18
A.4 Memory ON vs OFF: Cost and Consistency	18
A.5 Execution and Resource Metrics	20
A.6 Experiment Grid	20
A.7 SWE-bench Pro Pilot	21
A.8 Prompts	21

Table 3: Consolidated audited results on RTL-OPT [Lu et al., 2026], RTLRewriter-Bench [Yao et al., 2024], the custom-design PPA suite, and OpenTitan. Equiv. passed denotes equivalence-passing outputs; for RTLRewriter it also includes unchanged outputs accepted as trivially equivalent, while unproven outputs are excluded. RTL-OPT area wins and golden comparisons are reported among equivalence-passing outputs; its mean area gain averages all 38 designs with zero for invalid or non-improving outputs. RTLRewriter reports area wins over scored cases and geometric-mean cell/wire ratios. custom design suite gains retain only positive equivalence-accepted improvements. OpenTitan reports best-of-five strict-gate outcomes; accepted gains come from each design’s highest-scoring full-gate attempt, and WNS/TNS changes are in ns. RTL-OPT ChipMEM selects the best valid result per design across three result sets. Best values are bold.

(a) RTL-OPT					
Model	Syntax	Equiv. passed	Area < Suboptimal RTL / equiv. passed	Area < Golden RTL / equiv. passed	Avg. area improvement (%)
GPT-4o RTL-OPT	36/38	24/38	12/24 (50.0%)	10/24 (41.7%)	3.58%
GPT-4o-mini RTL-OPT	35/38	24/38	7/24 (29.2%)	6/24 (25.0%)	1.69%
DeepSeek V3 RTL-OPT	38/38	24/38	11/24 (45.8%)	10/24 (41.7%)	3.68%
DeepSeek R1 RTL-OPT	37/38	23/38	13/23 (56.5%)	9/23 (39.1%)	4.80%
GPT-5.5 EDA agent	37/38	33/38	14/33 (42.4%)	10/33 (30.3%)	6.18%
GPT-5.5* EDA agent + ChipMEM	37/38	33/38	19/33 (57.6%)	13/33 (39.4%)	8.79%
(b) RTLRewriter-Bench					
Model	Suite	Equiv. passed	Area wins	Avg. area (%) ↑	Cell/Wire ratio (× baseline)
GPT-5.5 EDA agent	Short	33/49	12/49	5.66%	0.927/0.923
GPT-5.5 EDA agent + ChipMEM	Short	37/49	15/49	8.69%	0.894/0.897
GPT-5.5 EDA agent	Long	2/5	1/5	6.85%	0.903/0.892
GPT-5.5 EDA agent + ChipMEM	Long	2/5	1/5	7.47%	0.872/0.858
(c) custom-design PPA audit					
Model	Equiv. passed	Area wins	Avg. area (%) ↑	Avg. power (%) ↑	Median timing (%)
GPT-5.5 EDA agent	8/20	3/20	0.1400%	0.3879%	0.0000%
GPT-5.5 EDA agent + ChipMEM	10/20	4/20	0.3101%	0.5211%	0.0000%
(d) OpenTitan ten-design PPA audit					
Model	Equiv. passed	Full PPA gate pass	Avg. area (%) ↑	Avg. power (%) ↑	WNS/TNS change (ns)
Qwen3.8-27B EDA agent	6/10	1/10	0.0519%	0.0000%	0.0/0.0
Qwen3.8-27B EDA agent + ChipMEM	6/10	4/10	0.2431%	2.5373%	0.0/0.0

A Appendix

Table 4: Five-pass CVDP training aggregates with 50 executions per setting.

Five-pass training aggregate				
Training task category	EDA agent	EDA agent + ChipMEM	Change	
CID012 stimulus	49/50 (98%)	48/50 (96%)	−2 pp	
CID013 checker	39/50 (78%)	42/50 (84%)	+6 pp	
Overall	88/100 (88%)	90/100 (90%)	+2 pp	

A.1 Per-Design PPA Results

Table 5: Per-design five-attempt PPA results for GPT-5.5 under *EDA agent* and *EDA agent + ChipMEM*. Pass@ k is cumulative full-synthesis completion (synth_status=1.0) within attempts 1– k . Area, power, and timing are positive design-level gains admitted by the documented equivalence check; all other outcomes contribute zero. Equivalence is reported as a binary PASS/FAIL.

Per-design five-attempt PPA evaluation (n = 20)											
Design	Model	Pass@1	Pass@2	Pass@3	Pass@4	Pass@5	Area Gain	Power Gain	Timing Gain	Synthesis	Equiv. pass
02_fifo_sync	EDA agent	0	0	1	1	1	0.0000%	0.0000%	0.0000%	1/5	FAIL
	EDA agent + ChipMEM	0	0	0	0	0	0.0000%	0.0000%	0.0000%	0/5	FAIL
03_rv32i_alu	EDA agent	1	1	1	1	1	0.0000%	0.0000%	0.0000%	2/5	FAIL
	EDA agent + ChipMEM	1	1	1	1	1	0.0000%	0.0000%	0.0000%	1/5	FAIL
09_apb_gpio	EDA agent	0	1	1	1	1	0.0567%	0.3937%	0.0000%	1/5	PASS
	EDA agent + ChipMEM	0	1	1	1	1	0.0000%	0.3937%	0.0000%	4/5	PASS
11_arbiter	EDA agent	0	1	1	1	1	0.0000%	1.4529%	9.5161%	2/5	PASS
	EDA agent + ChipMEM	0	0	1	1	1	0.0000%	0.0000%	0.0000%	3/5	FAIL
14_axi_adapter_rd	EDA agent	0	0	0	0	1	0.0000%	0.0000%	0.0000%	1/5	PASS
	EDA agent + ChipMEM	0	1	1	1	1	0.0000%	0.0000%	0.0000%	2/5	PASS
18_uart8receiver	EDA agent	1	1	1	1	1	0.0000%	0.0000%	0.0000%	2/5	FAIL
	EDA agent + ChipMEM	0	0	0	1	1	0.0000%	0.0000%	0.0000%	2/5	FAIL
ahb3lite_apb_bridge	EDA agent	0	0	0	1	1	0.0000%	0.0000%	0.0000%	2/5	FAIL
	EDA agent + ChipMEM	0	0	0	0	1	2.9366%	4.4379%	0.0000%	1/5	PASS
apb_spi	EDA agent	0	1	1	1	1	0.0000%	0.0000%	0.0000%	2/5	PASS
	EDA agent + ChipMEM	0	1	1	1	1	0.0000%	0.0000%	0.0000%	2/5	FAIL
axi_lite_interface	EDA agent	1	1	1	1	1	0.0000%	0.0000%	0.0000%	2/5	FAIL
	EDA agent + ChipMEM	1	1	1	1	1	0.0000%	0.0000%	0.0084%	3/5	PASS
bsg_ready_to_credit_flow_converter	EDA agent	0	0	1	1	1	2.6174%	5.4759%	2.2307%	2/5	PASS
	EDA agent + ChipMEM	0	0	0	0	0	2.6174%	5.4759%	2.2307%	0/5	PASS
bsg_serial_in_parallel_out_dynamic	EDA agent	0	0	1	1	1	0.0000%	0.0000%	0.0000%	3/5	PASS
	EDA agent + ChipMEM	0	0	1	1	1	0.0027%	0.0943%	0.0000%	3/5	PASS
cdc_fifo_master_hier	EDA agent	1	1	1	1	1	0.0000%	0.0000%	0.0000%	4/5	FAIL
	EDA agent + ChipMEM	0	0	0	1	1	0.0000%	0.0000%	0.0000%	2/5	FAIL
gpio	EDA agent	1	1	1	1	1	0.0000%	0.0000%	0.0000%	1/5	PASS
	EDA agent + ChipMEM	0	0	0	0	1	0.0000%	0.0000%	0.0000%	1/5	FAIL
hmac	EDA agent	0	0	0	0	0	0.0000%	0.0000%	0.0000%	0/5	FAIL
	EDA agent + ChipMEM	0	0	0	0	1	0.6454%	0.0194%	0.0000%	1/5	PASS
i2c	EDA agent	0	0	0	0	0	0.0000%	0.0000%	0.0000%	0/5	FAIL
	EDA agent + ChipMEM	0	0	0	1	1	0.0000%	0.0000%	0.0000%	1/5	FAIL
nand_flash_controller	EDA agent	0	1	1	1	1	0.0000%	0.0000%	0.0000%	2/5	FAIL
	EDA agent + ChipMEM	0	0	0	0	0	0.0000%	0.0000%	0.0000%	0/5	PASS
riscv_fetch	EDA agent	0	0	0	1	1	0.0000%	0.0000%	0.0000%	1/5	FAIL
	EDA agent + ChipMEM	1	1	1	1	1	0.0000%	0.0000%	0.0000%	4/5	PASS
sdram_axi	EDA agent	0	0	0	1	1	0.1264%	0.4348%	0.0000%	1/5	PASS
	EDA agent + ChipMEM	1	1	1	1	1	0.0000%	0.0000%	0.0000%	3/5	PASS
sram_ctrl	EDA agent	0	0	0	0	0	0.0000%	0.0000%	0.0000%	0/5	FAIL
	EDA agent + ChipMEM	0	1	1	1	1	0.0000%	0.0000%	0.0000%	3/5	FAIL
sv_cdc_fifo_master	EDA agent	1	1	1	1	1	0.0000%	0.0000%	0.0000%	3/5	FAIL
	EDA agent + ChipMEM	0	0	0	0	0	0.0000%	0.0000%	0.0000%	0/5	FAIL
Aggregate cumulative Pass@k (mean ± std across 20 designs)											
Model	Pass@1	Pass@2	Pass@3	Pass@4	Pass@5						
EDA agent	30.0±47.0%	50.0±51.3%	65.0±48.9%	80.0±41.0%	85.0±36.6%						
EDA agent + ChipMEM	20.0±41.0%	40.0±50.3%	50.0±51.3%	65.0±48.9%	80.0±41.0%						

A.2 Held-Out PPA Cohorts

We evaluate two disjoint cohorts of previously unseen RTL designs: cohort A contains 20 designs and cohort B contains 16 designs. Each design is evaluated once in cohort order. The evaluation begins with procedural skills distilled from earlier tool-evaluated PPA sessions, while library updates remain enabled. Consequently, a reused skill may originate from the initial library or from an earlier run in the ordered held-out evaluation. In the following tables, *Reused* denotes use of existing skill context, whereas *Mined* denotes creation of a skill from the current trajectory.

A.2.1 Per-design outcomes

Table 6 reports the outcome of each held-out evaluation. The per-design view distinguishes successful synthesis with available PPA measurements from runs that completed synthesis but did not produce sufficient metrics for PPA comparison.

Table 6: Per-design outcomes from the one-shot ChipMEM evaluation of held-out cohorts A and B using GPT-5.5. Synthesis scores indicate full (1.00) or partial (0.33) completion. PPA entries are raw relative changes; em dashes indicate unavailable metrics.

<i>Held-out A (n = 20)</i>						
Design	Memory action	Synthesis	Power	Area	Timing/WNS	PPA available
sync_fifo	Reused	0.33	-10.1% better	-7.2% better	+13.3% better	Yes
multibit_fifo_sync	Reused	0.33	—	—	—	No
credit_fifo	Reused	1.00	+0.1% same	+0.1% same	-6.4% worse	Yes
cdc_fifo	Reused	1.00	—	—	—	No
round_robin_arbiter	Reused	1.00	—	—	—	No
snapshot_arb	Reused	1.00	—	—	—	No
comparator	Reused	1.00	—	—	—	No
flow_reg	Reused	1.00	+0.0% same	+0.0% same	+0.0% same	Yes
scc	Reused	0.33	+0.0% same	-0.1% same	+0.0% same	Yes
rob	Reused	0.33	-1.6% better	-5.2% better	-15.6% worse	Yes
01_apb_protocol_slave	Reused	1.00	—	—	—	No
10_async_fifo	Reused	0.33	—	—	-100.0% worse	Yes
14_target_i2c	Reused	0.33	—	—	—	No
15_i2c_slave	Reused	0.33	+1.2% worse	-0.6% better	+0.8% better	Yes
16_ahb_slave	Reused	0.33	—	—	—	No
17_verilog_spi_master	Mined	0.33	—	—	—	No
26_cache_controller	Reused	1.00	—	—	—	No
29_pwm	Reused	0.33	-5.1% better	-14.5% better	+0.4% same	Yes
36_crc	Reused	1.00	—	—	—	No
40_lcd_controller	Reused	0.33	—	—	—	No
<i>Held-out B (n = 16)</i>						
Design	Memory action	Synthesis	Power	Area	Timing/WNS	PPA available
20_hazard_detection_unit_master	Reused	1.00	—	—	—	No
23_1_ethernet_mac_ip_64	Mined	0.33	-0.5% same	—	—	Yes
23_2_ethernet_mac_ip	Reused	1.00	—	—	—	No
24_1_pci_express_dma_if_axi	Mined	0.33	—	-0.1% same	+0.0% same	Yes
24_2_pci_express_dma_if_pcie	Mined	0.33	-1.8% better	+0.7% worse	+0.0% same	Yes
27_1_risc_v_core_fetch	Reused	1.00	—	—	—	No
27_2_risc_v_core_mmu	Reused	1.00	—	+0.0% same	+0.0% same	Yes
32_adc	Mined	0.33	—	+283.3% worse	-0.1% same	Yes
33_dac	Reused	1.00	—	—	—	No
35_ddr_controller	Mined	0.33	-3.6% better	+2.3% worse	-5.0% worse	Yes
45_fme_ctrl_vm	Mined	0.33	—	—	—	No
apbi2c	Mined	1.00	—	—	—	No
axi_master_design	Reused	1.00	—	—	—	No
pcie_controller_fsm	Mined	1.00	—	—	—	No
tri_mode_ethernet_mac	Reused	0.33	—	—	—	No
wb2axi	Mined	0.33	—	—	—	No

A.2.2 Skill inventory

Skills are grouped into three types. *Flow & Validation* covers tool setup, source ingestion, synthesis, constraints, and report collection. *Design Setup* captures compilation and synthesizability requirements specific to a design family. *RTL Optimization* records concrete RTL transformations and the conditions under which they produced positive, neutral, or negative results.

The retrieval columns report the number of designs in each cohort for which the agent received a skill, either as the selected skill or as additional context. A skill is counted at most once per design. Retrieval frequency therefore measures exposure to a skill, not whether it was applied or caused the resulting PPA outcome.

Table 7: Skill inventory and held-out retrieval frequency.

Skill	Skill summary	Cohort A retrievals	Cohort B retrievals
<i>Flow & Validation</i>			
sv-cdc-fifo-master	Require the intended CDC FIFO top, complete hierarchy, all clocks, and valid CDC constraints to synthesize before comparing area, power, or slack.	7	0
02-fifo-sync	Fix the source list and elaborate the intended FIFO top before changing RTL. Use the baseline only after synthesis produces timing, area, and power reports.	4	0
18-uart8receiver	Count the UART receiver run only after the intended top elaborates, maps, and produces timing, area, and power reports.	3	0
ahb3lite-apb-bridge	A frontend warning does not necessarily mean the bridge failed. Check that the AHB-to-APB top and its hierarchy elaborate before stopping the run.	2	0
nand-flash-controller	A missing-top error is a source-ingestion problem, not a PPA result. Fix the flash-controller filelist and elaboration before changing RTL.	2	0
streaming-datapath	Use a complete source list and fixed constraints, then write timing, area, and power reports to known files before comparing datapath revisions.	0	2
dma-engine	Resolve the DMA top, complete source list, constraints, and report paths before modifying datapath or control RTL.	0	1
03-rv32i-alu	Reading the HDL is not enough. The intended processor top must elaborate and synthesize before the ALU has usable QoR data.	0	0
apb-spi	If the APB-SPI top is missing after the HDL read, check the exact module name, source list, and child-module coverage before editing RTL.	0	0
memory-controller	Use the controller's PPA results only after the top and full hierarchy synthesize under a defined constraint set and the reports contain numeric results.	0	0
serial-interface-controller	When the serial-controller top is missing after the HDL read, use the parser diagnostics to fix the source list, search path, or language mode.	0	0
<i>Design Setup</i>			
bsg-ready-to-credit-flow-converter	Keep the original source order, include paths, defines, and synthesis guards. Changing the preprocessor context can hide the converter top or select the wrong RTL branch.	1	0
gpio	Supply the assertion macros or synthesis-only stubs expected by the GPIO sources. Do not change functional RTL to work around missing verification headers.	1	0
i2c	Use the intended assertion headers, defines, include directories, and synthesis macro branch. Unresolved macros can prevent the IC top from parsing.	1	0
sram-ctrl	Preserve package and source order, include directories, defines, and assertion macros in the SRAM controller filelist. Local shims can change preprocessing or leave the hierarchy incomplete.	1	0
fsm-controller	Code asynchronous reset and synchronous initialization as separate conditions. Only the true asynchronous reset belongs in the reset branch.	0	0

Continued on next page

Table 7 (continued)

Skill	Skill summary	Cohort A retrievals	Cohort B retrievals
<i>RTL Optimization</i>			
<code>riscv-fetch</code>	Delete only state proven constant or unobservable. Preserve redirects, stalls, flushes, skid-buffer behavior, PC updates, reset behavior, and cycle latency.	2	3
<code>cdc-fifo-master-hier</code>	Synthesize the complete FIFO hierarchy with no black boxes. Pointer or register-enable rewrites increased mapped cost and must preserve the CDC, flag, and memory behavior.	3	1
<code>11-arbiter</code>	Recoding the round-robin state-update logic increased mapped cost. Limit changes to local common-term or decode cleanup, then compare post-synthesis QoR.	3	0
<code>axi-lite-interface</code>	Explicit register enables, hold-mux rewrites, and idle-data gating were neutral or worse after mapping and can change AXI-Lite-visible behavior.	3	0
<code>dma-interface</code>	Derive load enables for wide DMA registers from existing transfer or state conditions. Preserve reset values and interface latency, then measure switching, area, and slack after mapping.	0	3
<code>09-apb-gpio</code>	Vectorizing per-bit GPIO logic or factoring APB address decode can map worse. Preserve the register map, read/write behavior, interrupts, and cycle timing.	2	0
<code>14-axi-adapter-rd</code>	Parameter-controlled tie-offs on disabled AXI read paths produced no QoR gain, likely because synthesis had already removed the inactive logic.	2	0
<code>bsg-serial-in-parallel-out-dynamic</code>	A one-entry FIFO controller can be recoded only if depth, valid/occupancy behavior, handshake timing, latency, throughput, and parameter behavior stay unchanged.	1	0
<code>hazard-detector</code>	Factor repeated opcode and register-decode terms and use direct equality comparisons in combinational hazard logic. Prove equivalence and compare mapped QoR.	0	1
<code>priority-encoder</code>	Replacing a compact priority or comparator cone with enumerated case or threshold logic increased mapped area or power. Keep the RTL form the mapper handles better.	0	1
<code>bus-bridge</code>	Forcing combinational bridge signals to zero or adding speculative enables increased logic and hurt slack. Keep gating only when mapped power, area, and timing improve.	0	0
<code>hmac</code>	Build only the digest or message word selected in the current cycle instead of a full-width combinational bus. Recheck timing because the narrower mux cone may have different depth.	0	0
<code>protocol-bridge</code>	Remove identical-arm ternaries, redundant self-assignments, combinational self-feedback, and arithmetic identities in the bridge control logic. Resynthesize each change separately.	0	0
<code>sdram-axi</code>	Operand or payload gating on AXI/SDRAM data paths added muxing and could raise area or dynamic power. Keep it only when mapped QoR improves under the same activity assumptions.	0	0

A.3 OpenTitan Ten-Design PPA Audit

Table 8: Complete per-design cumulative strict-gate audit for the OpenTitan experiment. Attempts 1–5 are complete for all ten designs in both modes. All runs use Qwen3.8-27B and the same frozen harness. A pass requires the strict equivalence and OpenROAD gate to succeed.

<i>OpenTitan per-design cumulative binary audit ($n = 10$)</i>						
Design	Model	Pass@1	Pass@2	Pass@3	Pass@4	Pass@5
aon_timer	EDA agent	1	1	1	1	1
	EDA agent + ChipMEM	0	0	1	1	1
rv_timer	EDA agent	0	0	0	0	0
	EDA agent + ChipMEM	0	0	0	0	0
pattgen	EDA agent	0	0	0	0	0
	EDA agent + ChipMEM	0	0	0	1	1
i2c	EDA agent	0	0	0	0	0
	EDA agent + ChipMEM	0	0	0	0	0
spi_host	EDA agent	0	0	0	0	0
	EDA agent + ChipMEM	0	0	1	1	1
hmac	EDA agent	0	0	0	0	0
	EDA agent + ChipMEM	0	0	0	0	0
dma	EDA agent	0	0	0	0	0
	EDA agent + ChipMEM	0	0	0	0	0
keymgr	EDA agent	0	0	0	0	0
	EDA agent + ChipMEM	0	0	1	1	1
aes	EDA agent	0	0	0	0	0
	EDA agent + ChipMEM	0	0	0	0	0
kmac	EDA agent	0	0	0	0	0
	EDA agent + ChipMEM	0	0	0	0	0
<i>Aggregate cumulative Pass@k (mean $\pm$ sample std across ten designs)</i>						
EDA agent		10.0% $\pm$ 31.6%	10.0% $\pm$ 31.6%	10.0% $\pm$ 31.6%	10.0% $\pm$ 31.6%	10.0% $\pm$ 31.6%
EDA agent + ChipMEM		0.0% $\pm$ 0.0%	0.0% $\pm$ 0.0%	30.0% $\pm$ 48.3%	40.0% $\pm$ 51.6%	40.0% $\pm$ 51.6%

A.4 Memory ON vs OFF—Full Cost, Success, and Consistency Data

Figure 4 summarizes the interaction-cost comparison; this section reports the complete task- and attempt-level evidence. The completed legacy run mined a skill after every memory-on session, including all 45 failed sessions, so these results are descriptive of this run rather than a clean evaluation of the current PASS-only learning rule.

A.4.1—Per-design success

Table 9: Per-design success. EDA Agent + ChipMEM produces passing, equivalence-verified edits on 4/10 designs versus 1/10 for EDA Agent, including three designs (pattgen, spi_host, and keymgr) on which the EDA Agent fails every attempt.

Design	OFF passes (of 5)	ON passes (of 5)	Note
aon_timer	2	1	
pattgen	0	2	Unlocked by memory
spi_host	0	1	Unlocked by memory
keymgr	0	1	Unlocked by memory
rv_timer	0	0	
i2c	0	0	
hmac	0	0	
dma	0	0	
aes	0	0	
kmac	0	0	
Total	2 passes; 1/10 designs	5 passes; 4/10 designs	

A.4.2—Overall interaction cost

Table 10: Overall interaction cost across 50 sessions per mode. Output tokens and session wall time favor EDA Agent; the increase is consistent with memory writes, reflection, and longer-running successful sessions. Wall time is cumulative session duration, not parallel-run makespan.

Metric	OFF total	ON total	OFF mean	ON mean	Δ mean
Total tokens	15.357M	14.834M	307.1k	296.7k	−3.4%
Input tokens	14.516M	13.530M	290.3k	270.6k	−6.8%
Output tokens	0.841M	1.304M	16.8k	26.1k	+55.0%
LLM calls	1,468	1,303	29.36	26.06	−11.2%
Tool calls	1,372	1,166	27.44	23.32	−15.0%
Tool time	350.19 min	308.14 min	7.00 min	6.16 min	−12.0%
Wall time	2,048.4 min	2,353.2 min	41.0 min	47.1 min	+14.9%

Medians. LLM calls: OFF 40 (the budget cap), ON 27. Tool calls: OFF 36.5, ON 18. Total tokens: OFF 373.6k, ON 379.2k.

A.4.3—Attempt-level paired comparison

Table 11: Pairwise deltas over 50 design-and-repeat matched attempts. Efficiency deltas are directionally favorable to EDA Agent + ChipMEM, but clustered confidence intervals cross zero except for output tokens, which increase with memory. Primary full-gate outcome summaries appear in Table 9.

Metric	ON lower	Tied	ON higher	Cluster-bootstrap 95% CI of mean Δ
Input tokens	31	0	19	[−58,049, +13,391]
Output tokens	18	0	32	[+318, +20,726]
Total tokens	27	0	23	[−54,672, +29,844]
LLM calls	25	20	5	[−7.04, +0.30]
Tool calls	29	10	11	[−8.74, +0.16]
Tool seconds	25	0	25	[−141.1, +19.6]
Wall seconds	19	0	31	[−196.5, +1,062.8]

A.4.4—Per-design consistency

Table 12: Direction of the per-design mean change across the ten OpenTitan designs.

Metric	ON better	Tie	ON worse
Total tokens	6/10	0	4/10
LLM calls	7/10	2/10	1/10
Tool calls	8/10	0	2/10
Tool time	5/10	0	5/10

Note. The LLM-call ties are `keymgr` and `aes`; both reach the 40-call cap in both modes.

A.4.6—Matched case study: `aon_timer`

Table 13: On the only design with at least one full-gate pass in both modes, EDA Agent + ChipMEM uses roughly half the token and call cost and one-third less session wall time. Values are per-attempt means.

Metric	OFF	ON	Δ
Total tokens	321.5k	156.3k	−51%
Input tokens	292.6k	138.6k	−53%
LLM calls	26.6	13.6	−49%
Tool calls	24.0	12.6	−48%
Tool time	40.2 s	22.7 s	−44%
Wall time	1,303.5 s	858.0 s	−34%

A.5 Execution and Resource Metrics

Table 14: Execution audit for the 100 Qwen3.8-27B canonical sessions underlying Table 3 and Appendix Table 8: ten designs, five attempts, and two modes. Continuous entries are mean $\pm$ sample standard deviation. Across the 50 sessions per mode, *EDA agent*/*EDA agent + ChipMEM* used 15.357M/14.834M total tokens, 34.14/39.22 cumulative session-hours, 1468/1303 LLM calls, and 1372/1166 tool calls. All canonical rows and transcripts were read from the completed Brev snapshots. Baseline/candidate OpenROAD times are reported only where gate metrics were recovered; their available n is shown. Four source-migrated attempt-1 transcripts do not reconstruct the retained canonical tool-time field exactly, so the table uses the preserved canonical values. The OpenTitan canonical schema did not preserve a retry counter; aborted-session counts were recovered from the exact transcript flags. Cached-token and reasoning-token counts were unavailable and are not reported as zero.

OpenTitan latency, tokens, and interaction counts											
Mode	Attempt	n	Wall (min)	Tool (min)	CPU (s)	Input kTok	Output kTok	Total kTok	LLM calls	Tool calls	
EDA agent	1	10	66.1 $\pm$ 64.7	8.87 $\pm$ 11.31	0.36 $\pm$ 0.14	287.2 $\pm$ 155.8	18.0 $\pm$ 16.9	305.2 $\pm$ 159.9	29.1 $\pm$ 14.2	26.9 $\pm$ 15.1	
EDA agent + ChipMEM	1	10	52.4 $\pm$ 37.9	6.92 $\pm$ 8.24	5.23 $\pm$ 3.23	274.8 $\pm$ 171.8	17.4 $\pm$ 23.7	292.2 $\pm$ 182.2	26.9 $\pm$ 14.8	25.0 $\pm$ 15.1	
EDA agent	2	10	33.0 $\pm$ 23.8	5.96 $\pm$ 7.50	0.19 $\pm$ 0.11	274.9 $\pm$ 147.6	14.2 $\pm$ 8.7	289.1 $\pm$ 146.1	27.6 $\pm$ 13.4	26.0 $\pm$ 14.2	
EDA agent + ChipMEM	2	10	43.4 $\pm$ 26.2	5.99 $\pm$ 7.60	4.86 $\pm$ 2.99	257.2 $\pm$ 161.5	23.2 $\pm$ 26.1	280.4 $\pm$ 175.4	26.3 $\pm$ 14.9	24.4 $\pm$ 14.9	
EDA agent	3	10	31.3 $\pm$ 23.1	6.60 $\pm$ 7.56	0.20 $\pm$ 0.12	284.2 $\pm$ 147.4	12.5 $\pm$ 12.3	296.7 $\pm$ 147.9	29.7 $\pm$ 13.3	28.3 $\pm$ 14.0	
EDA agent + ChipMEM	3	10	46.4 $\pm$ 30.1	5.93 $\pm$ 6.65	5.27 $\pm$ 3.28	270.3 $\pm$ 150.8	26.8 $\pm$ 23.2	297.2 $\pm$ 160.6	26.4 $\pm$ 13.3	23.4 $\pm$ 14.4	
EDA agent	4	10	39.0 $\pm$ 27.9	6.47 $\pm$ 8.56	0.21 $\pm$ 0.12	314.9 $\pm$ 156.4	21.7 $\pm$ 34.3	336.6 $\pm$ 171.6	31.4 $\pm$ 13.9	29.1 $\pm$ 14.1	
EDA agent + ChipMEM	4	10	45.8 $\pm$ 29.2	5.61 $\pm$ 6.76	5.68 $\pm$ 3.41	265.3 $\pm$ 168.4	28.2 $\pm$ 28.4	293.5 $\pm$ 187.5	25.3 $\pm$ 13.6	22.4 $\pm$ 13.4	
EDA agent	5	10	35.4 $\pm$ 22.5	7.11 $\pm$ 8.26	0.22 $\pm$ 0.14	290.3 $\pm$ 157.1	17.8 $\pm$ 19.8	308.1 $\pm$ 164.5	29.0 $\pm$ 14.5	26.9 $\pm$ 15.0	
EDA agent + ChipMEM	5	10	47.2 $\pm$ 28.9	6.36 $\pm$ 7.83	6.19 $\pm$ 3.93	285.3 $\pm$ 159.5	34.8 $\pm$ 31.1	320.1 $\pm$ 173.5	25.4 $\pm$ 12.2	21.4 $\pm$ 13.1	
OpenTitan gate coverage and memory-control activity											
Mode	Attempt		Baseline ORFS (min)	Candidate ORFS (min)	Gate n	Equiv. pass	Pass/Fail	Aborted	Bayes updates	Nudges	Retrieved/Mined
EDA agent	1		28.9 $\pm$ 49.9	22.8 $\pm$ 33.9	8/10	5/10	1/9	6/10	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0 / 0
EDA agent + ChipMEM	1		38.0 $\pm$ 56.4	14.9 $\pm$ 13.4	6/10	3/10	0/10	7/10	25.0 $\pm$ 15.1	1.6 $\pm$ 1.4	0.0 $\pm$ 0.0 / 10
EDA agent	2		32.3 $\pm$ 48.8	13.6 $\pm$ 12.9	8/10	5/10	0/10	6/10	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0 / 0
EDA agent + ChipMEM	2		32.3 $\pm$ 48.8	13.5 $\pm$ 12.7	8/10	3/10	0/10	6/10	24.4 $\pm$ 14.9	2.7 $\pm$ 0.7	0.1 $\pm$ 0.3 / 10
EDA agent	3		29.3 $\pm$ 49.9	11.3 $\pm$ 13.1	8/10	6/10	1/9	7/10	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0 / 0
EDA agent + ChipMEM	3		29.0 $\pm$ 46.7	12.1 $\pm$ 14.0	9/10	6/10	3/7	4/10	23.4 $\pm$ 14.4	2.7 $\pm$ 0.7	0.2 $\pm$ 0.4 / 10
EDA agent	4		29.0 $\pm$ 46.7	12.0 $\pm$ 14.2	9/10	6/10	0/10	6/10	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0 / 0
EDA agent + ChipMEM	4		32.3 $\pm$ 48.8	12.4 $\pm$ 12.5	8/10	4/10	1/9	6/10	22.4 $\pm$ 13.4	2.8 $\pm$ 0.6	0.2 $\pm$ 0.4 / 10
EDA agent	5		32.3 $\pm$ 48.8	11.8 $\pm$ 13.4	8/10	5/10	0/10	7/10	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0 / 0
EDA agent + ChipMEM	5		32.3 $\pm$ 48.8	10.9 $\pm$ 12.8	8/10	3/10	1/9	4/10	21.4 $\pm$ 13.1	2.5 $\pm$ 0.8	0.3 $\pm$ 0.5 / 10

A.6 Experiment Grid

The main controlled grids contain 200 proprietary PPA executions, 100 OpenTitan executions, and 240 CVDP simulation executions.

A.7 SWE-bench Pro Pilot

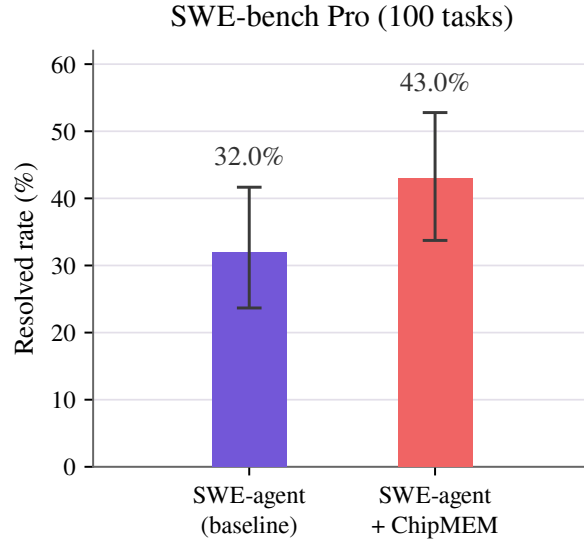


Figure 6: SWE-bench Pro resolved rate in a 100-task pilot per mode. Error bars show Wilson 95% confidence intervals. SWE-agent + ChipMEM resolves 43/100 tasks compared with 32/100 for the SWE-agent baseline, an absolute difference of 11 percentage points.

Table 15: Aggregate SWE-bench Pro outcomes for the 100-task pilot. The preserved pilot summary does not include matched per-task outcomes, so we do not report a paired significance test.

Mode	Tasks	Resolved	Unresolved	Rate	95% CI
SWE-agent baseline	100	32	68	32.0%	[23.7, 41.7]
SWE-agent + ChipMEM	100	43	57	43.0%	[33.7, 52.8]

A.8 Prompts

The prompts are grouped by the model and reported experiment in which they were used. Task-specific values and session data were inserted at runtime.

EDA agent with GPT-5.5. The following distillation prompts produced procedural skills for the reported GPT-5.5 EDA-agent experiments.

EDA Agent GPT-5.5 Skill-Distillation Prompt

You distill one agent run into a reusable learned skill.
 Use only the supplied session and measured reward. Return strict JSON:

```
{
  "name": "generic-lowercase-skill-name",
  "description": "one-line description",
  "body": "Markdown with ## DO and ## AVOID sections",
  "metadata": {"failure_modes": "short summary"}
}
```

Keep rules generic and evidence-based. A failed or regressed action belongs in AVOID. Successful measured actions belong in DO. Do not include instance-specific paths or secrets.

EDA Agent GPT-5.5 Distillation Input Template

```
AGENT TYPE: {{ agent_type }}
INITIAL REQUEST: {{ initial_request }}
REWARD: score={{ reward_score }}, success={{ reward_success }},
        components={{ reward_components }}, failure_class={{ failure_class }}
{% if existing_skill_body %}

EXISTING SKILL TO UPDATE:
{{ existing_skill_body }}
{% endif %}

SESSION:
{{ session_transcript }}
```

EDA agent with Qwen3.8-27B on OpenTitan. The following agent and distillation prompts were used only for the reported OpenTitan experiment.

EDA Agent Qwen3.8-27B OpenTitan Prompt

You are an autonomous engineer. Solve the task using the available tools.
When done, reply with your final summary instead of a tool call.

Optimize OpenTitan IP {{ design }} for lower physical area and power without changing behavior or ports. You may edit only the files returned by list_files. First inspect the RTL and run orfs_synth for a baseline. Make conservative changes, rerun orfs_synth after each change, and keep only measured improvements. A strict equivalence check and full ORFS flow will grade the result.

Qwen3.8-27B OpenTitan Distillation Prompt

Distill exactly one reusable flat PPA skill from this scored OpenTitan PASS trajectory. Return only concise DO and AVOID lines.

The measured session trajectory was supplied to the distiller together with this fixed instruction. Memory OFF and Memory ON used the same agent prompt, model settings, tools, and execution budget. Memory ON additionally received the retrieved procedural skills.